

Robustness, stress sweep, longevity, controller comparison: what it does and why it is correct

July 2026

Summary. Four controllers on the same stressed day (`mdot_scale`= 0.35): KPC reaches 97.56 % worst-consumer demand met and the fair iterated baseline 97.74 %, a tie at the flow-limited capacity ceiling; both sit above the exact-model NMPC (95.49 %, about 62× slower) and the Jacobian-LMPC (87.34 %). With flow headroom KPC holds 100 % at every capacity point from 0.5 up, while the iterated rollout falls to 93.2 % at full flow (gap 6.78 pp). A continuous 90-day run shows zero comfort drift and 1 fallback step in 8640. All eight validator invariants pass (Section 4).

1 What this does

Three closed-loop studies plus one coded gate. (1) A four-controller head-to-head on one severe stressed day (`mdot_scale`= 0.35: 35 % of nominal flow; 24 hours; $N_p = 12$), one shared setup so only the controller differs. (2) A capacity sweep of the two Koopman controllers, `mdot_scale`= 0.35 to 1.00. (3) One continuous 90-day loop at half capacity. `tests/validate_robustness.m` asserts eight invariants (C1–C6, L1–L2) on the saved files without re-simulating.

Comfort metric everywhere: the daily worst-consumer energy ratio,

$$\text{met}_i = 100 \frac{\sum_k c_i[k]}{\sum_k d_i[k]}, \quad \text{headline} = \min_i \text{met}_i,$$

c_i delivered, d_i demanded heat; a daily energy metric lets a controller lead and recover around the peak.

The four controllers: **KPC** (direct multi-step Koopman prediction, one convex QP per step, this work); the **iterated Koopman-LMPC** (same learned 47-feature lift, iterated one-step model, one QP); a **Jacobian-LMPC** (re-linearise the nonlinear plant every step, one QP, $H = 2J^\top J + 10^{-6}I$); and a full **NMPC** (`fmincon` on the exact plant),

$$\min_{T^{(0,s)}, \xi} \sum_{i,h} (C_{i,h}(T^{(0,s)}, \xi) - d_{i,h})^2, \quad r_q = m_d + N\xi, \quad N = \text{null}([M_s; M_r]),$$

where C re-integrates the nonlinear plant, m_d are the nominal per-edge flows and ξ the free null-space coordinates, so Kirchhoff mass conservation stays exact inside the optimiser.

All four share plant, scenario, warmup, rate caps and the demand-adequacy flow floor; the baselines get KPC's `quadprog` tolerances. The NMPC budget (`MaxIterations` 300, `MaxFunctionEvaluations` 3000) lets `fmincon` stop on its own tolerance, so its solve times and its 10 of 96 exitflag -2 steps are genuine (it applies the last iterate there). Two asymmetries remain, both favouring the baselines: they keep the wider actuator box (source floor 14 °C, flow floor 0.3× nominal, against the tuned 17.69 °C and 0.5694×), and their floor uses the current-step

demand where KPC uses the horizon maximum. The shared setup (`fair_comparison`, see `comparison_validation.pdf`) equalises these: NMPC lands at 94.6% and the Jacobian-LMPC at 91.6%, Koopman rows unchanged. That run also scores solver status more strictly, counting only certified-optimal steps: there NMPC is first-order optimal on 29 of 96 and about $81\times$ KPC’s solve time, against the converged counts and $\sim 62\times$ in this part-4 table.

controller	worst-met [%]	unmet [kWh]	median solve	converged
KPC (this work)	97.56	1.03	0.97 s	96 / 96
Koopman-LMPC (iterated)	97.74	0.95	~ 0.6 s	96 / 96
NMPC (exact model)	95.49	2.93	60.2 s (max 259 s)	86 / 96
Jacobian-LMPC	87.34	16.28	13.3 s	96 / 96

Table 1: Severe stressed day, `mdot_scale`= 0.35. KPC clears the 95% bar, is feasible on all 96 QP steps at 966 ms median (0.107% of the $T_s = 900$ s budget), and beats both plant-model controllers; NMPC is about $62\times$ slower and fails to converge on 10 of 96 steps; the Jacobian-LMPC falls below the bar.

One wrinkle. At the `mdot`= 0.35 ceiling the two Koopman controllers tie (97.56 vs 97.74), where network flow limits delivery, so C2 is the comfort bar plus beating both plant models and the direct-versus-iterated separation shows up in the sweep (C6). KPC holds 100.0% at every point from 0.5 up; the iterated rollout drops to 99.38, 97.12 and 93.22% at `mdot` 0.65, 0.80 and 1.00, below the 95% bar at full flow (gap 6.78 pp).

Longevity. One continuous closed-loop run at `mdot_scale`= 0.50 for 90 days (8640 steps). Drift +0.000 pp (both days at 100.00%), per-day median solve 0.9–1.9 s, and exactly 1 non-converged step in 8640 (0.0116%), after which the controller holds its previous command and recovers. The compute margin stays about 0.1% of the sample budget; district-size scaling runs live outside this package.

Code layout

file	role
<code>../visualizations/4_comparison/run_comparison.m</code>	the four-controller stressed day
<code>../visualizations/4_comparison/baseline_cl_run.m</code>	shared closed-loop runner for the baselines
<code>../visualizations/4_comparison/run_stress_sweep.m</code>	the capacity sweep 0.35–1.00
<code>run_longevity.m</code>	the continuous 90-day run
<code>demo_robustness.m</code>	renders <code>figures/longevity.pdf</code> from the saved run
<code>tests/validate_robustness.m</code>	the eight coded invariants (C1–C6, L1–L2)

2 How to reproduce

Fast path (seconds, since the expensive runs ship as `.mat`), from the repo root in MATLAB: `startup; viz_compare; run('visualizations/4_comparison/viz_stress_sweep.m');` `demo_robustness; validate_robustness` (or `run_all_tests`), ending `=== Part 4 validation: all 8 invariants passed ===`. Full regeneration (order matters, NMPC dominates the cost): see README.

3 The figures

3.1 Four-controller comparison (viz_compare.pdf)

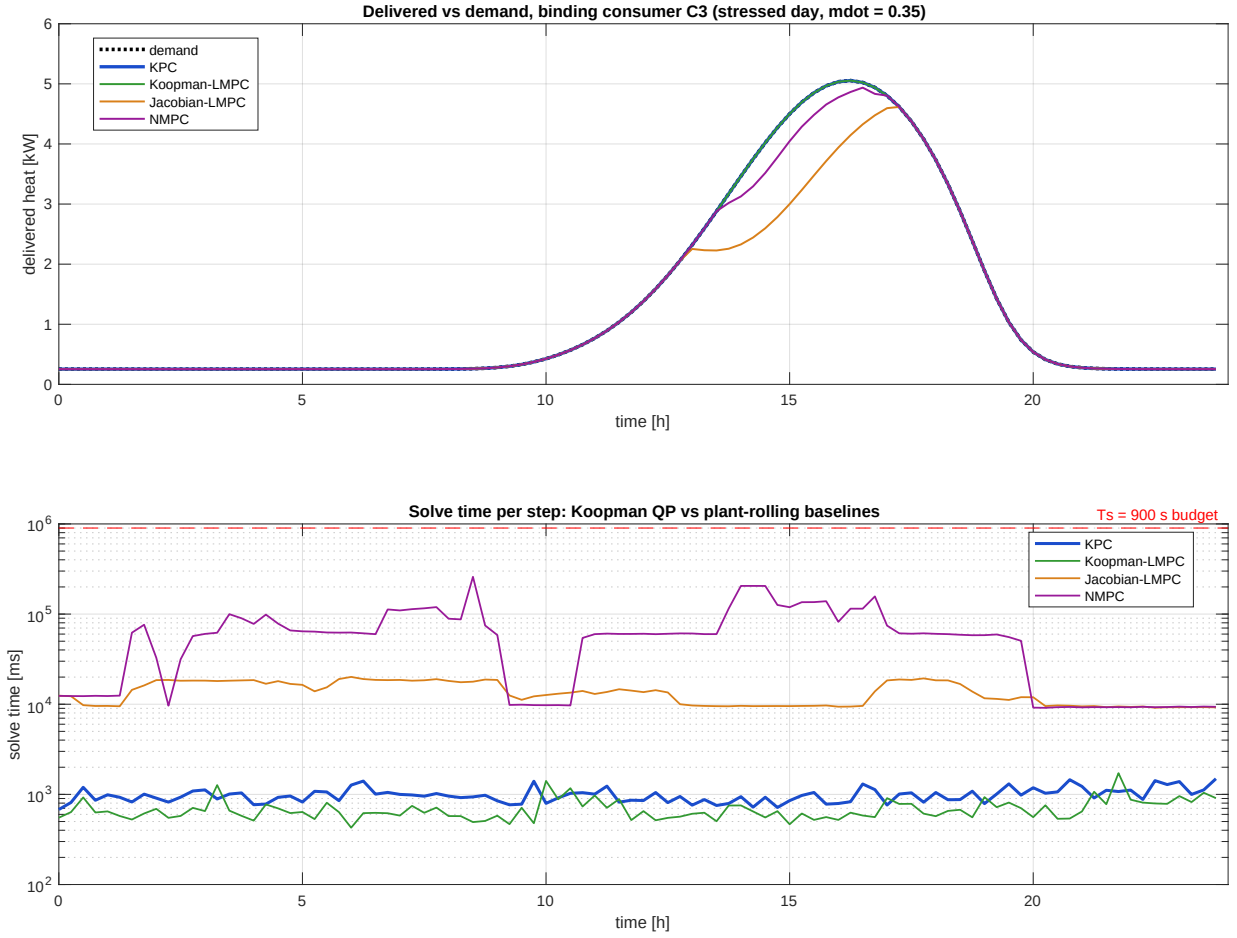


Figure 1: Stressed day: demand tracking at the binding consumer C3 and per-step solve times. Both Koopman controllers hug the demand curve; NMPC lands just under at $\sim 60\times$ the solve time; the Jacobian-LMPC falls short; all solve times sit under the $T_s = 900$ s budget.

Tracking is drawn at consumer C3, the worst-served across the four (it sets the Jacobian headline; the other three worst-met numbers are set by consumer C5). At $\dot{m} = 0.35$ delivery is flow-limited, so both Koopman controllers riding the ceiling confirms the lift carries the delivery-relevant dynamics.

3.2 Capacity sweep (stress_sweep.pdf)

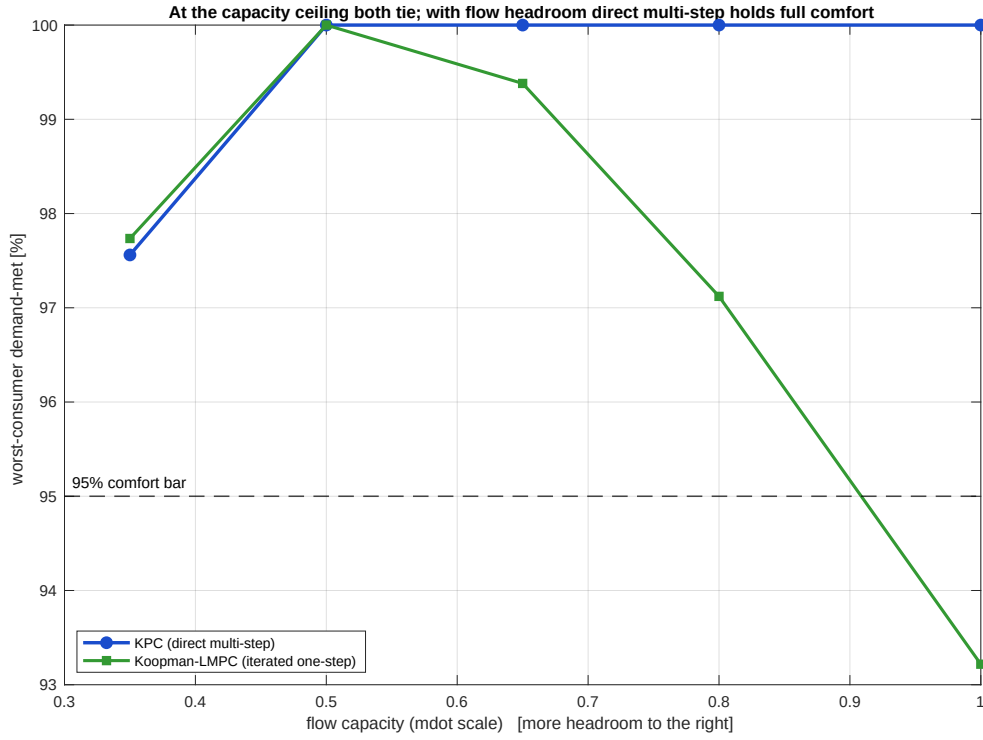


Figure 2: Capacity sweep of the two Koopman controllers. The two nearly coincide at the tight-capacity ceiling (97.56 against 97.74); from $\text{mdot}=0.5$ up KPC stays flat at 100% while the iterated baseline falls away and crosses the 95% bar before full flow.

At the ceiling flow binds, not prediction, so the tie is expected. With headroom the iterated one-step model compounds error over the rollout, while the direct map is fit at the horizons it is used at.

3.3 90-day longevity (figures/longevity.pdf)

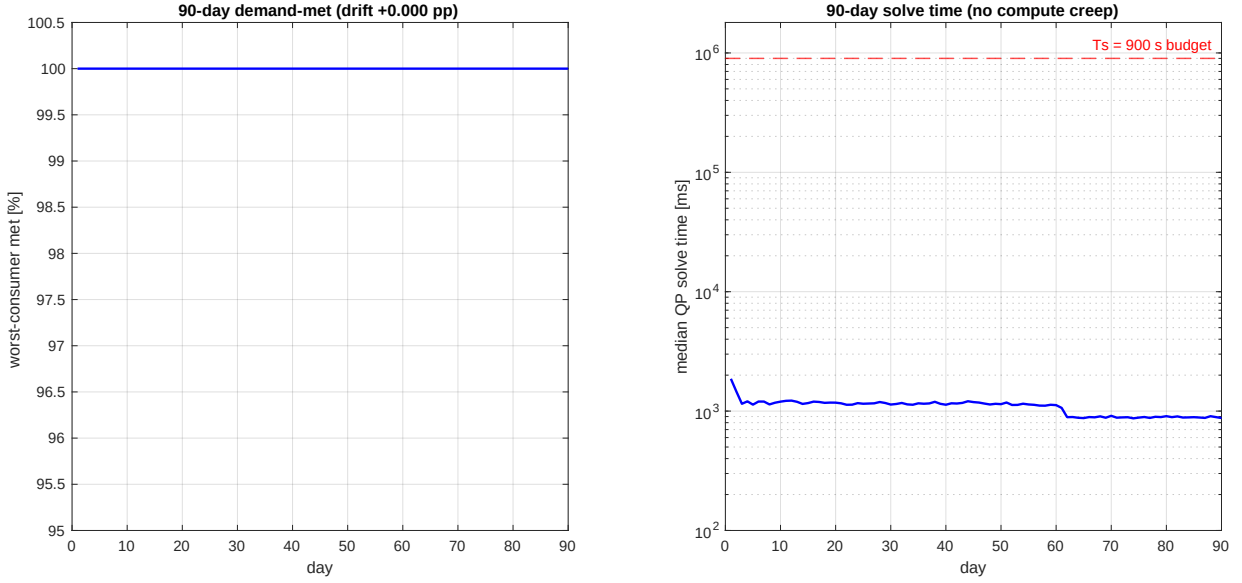


Figure 3: Continuous 90-day run: per-day worst-consumer met pinned at 100 % (drift +0.000 pp) and per-day median solve far below budget with no upward creep.

Two solve-trace features are wall-clock effects, not controller events: the first days start near 1.9s and step down to about 0.9s near day 60 (the demand repeats identically, so the QP is unchanged).

4 The automated checks

`tests/validate_robustness.m` loads only the saved result files (`comparison.mat`, `stress_sweep.mat`, `results/longevity.mat`) and asserts eight invariants without re-simulating (the labels C1–C6, L1–L2 are not the consumer names C1–C5).

#	asserts	threshold	observed
C1	KPC QP feasible on every comparison step	all steps	96 / 96
C2	comfort bar; beats both plant-model baselines	$\geq 95\%$	97.56 (NMPC 95.49, Jac. 87.34)
C3	KPC speedup versus NMPC (median over median)	$\geq 10\times$	$\sim 62\times$
C4	KPC median solve as share of the T_s budget	$< 5\%$	0.107 %
C5	saved run reproduces the headline comfort numbers	± 0.2 pp	97.561 / 97.735
C6	sweep: ≥ 95 everywhere, ≥ 99.9 with headroom, gap	≥ 2 pp	min 97.56; gap 6.78 pp
L1	90-day worst-consumer drift	≤ 0.5 pp	+0.000 pp
L2	90-day infeasible-step rate	$\leq 0.1\%$	0.0116 %

Table 2: The eight coded invariants in `validate_robustness.m`.

C1 feasibility recovers by construction through the priced soft-constraint slacks (Kerrigan and Maciejowski, 2000). C2 is the headline in honest form: clear the bar and beat both plant models; the Koopman tie at the ceiling is explained, not asserted away, and the separation is tested in C6. C3–C4 are the compute case, margins loose enough to hold across machines. C5 is provenance: the shipped `comparison.mat` must reproduce the reported numbers. L1–L2 are the deployment claim. The bars are acceptance floors, not fitted to the result; the threshold-versus-observed columns above show how loose they are.